

Model structure

MIPT

2025

Model selection

First level: select optimal parameters:

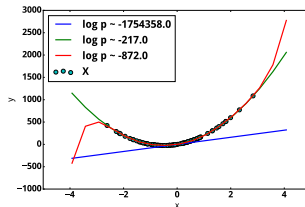
$$\mathbf{w} = \arg \max \frac{p(\mathcal{D}|\mathbf{w})p(\mathbf{w}|\mathbf{h})}{p(\mathcal{D}|\mathbf{h})},$$

Second level: select model optimizing Evidence:

$$p(\mathcal{D}|\mathbf{h}) = \int_{\mathbf{w}} p(\mathcal{D}|\mathbf{w})p(\mathbf{w}|\mathbf{h})d\mathbf{w}.$$



Model selection scheme

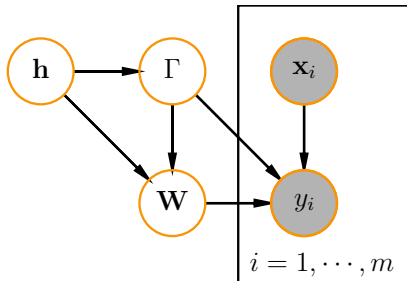


Example

Prior

Definition

Prior for parameters w and structure Γ of the model f is a distribution $p(W, \Gamma | h) : \mathbb{W} \times \mathbb{\Gamma} \times \mathbb{H} \rightarrow \mathbb{R}^+$, where $\mathbb{W}$ is a parameter space, $\mathbb{\Gamma}$ is a structure space.



Definition

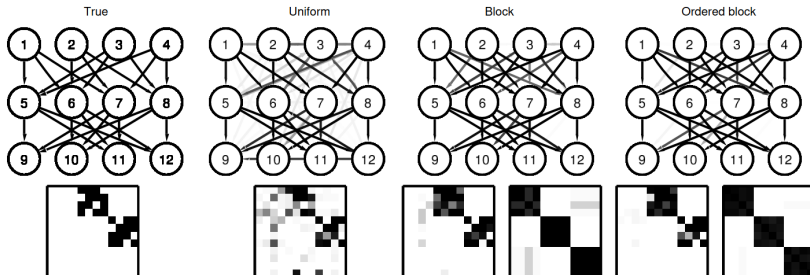
Hyperparameters $h \in \mathbb{H}$ of the models are the parameters of $p(w, \Gamma | h)$ (parameters of prior f).

Model f is assigned by the following values:

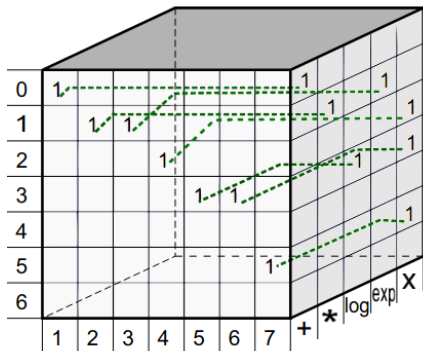
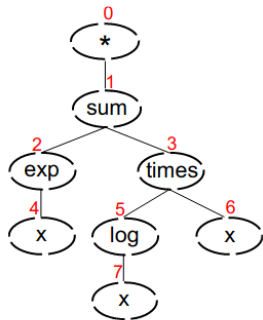
- **Parameters** $w \in \mathbb{W}$ set the superposition of submodels f_v .
- **Structure parameters** $\Gamma = \{\gamma^{j,k}\}_{(j,k) \in E} \in \mathbb{\Gamma}$ set importance of each submodel f_v .
- **Hyperparameters** $h \in \mathbb{H}$ set prior distribution .
- **Metaparameters** $\lambda \in \mathbb{A}$ set optimization function.

How can we assign model structure?

Example: Bayesian networks



Example: prediction of ranking functions



$$f = \exp(x) + (\log x)x$$

Трехиндексная матрица связей Z_f дерева Γ_f

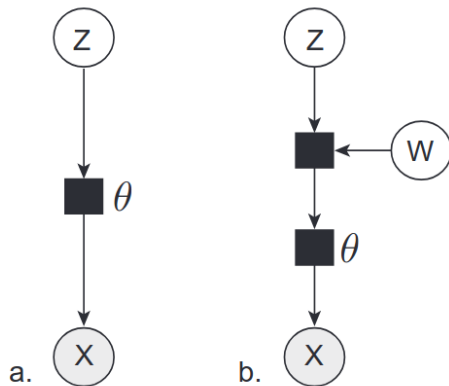
- вершины дерева пронумерованы;
- первые два индекса — номера вершин в ребре;
- третий индекс — выбранная элементарная функция на конце ребра.

Can we treat model structure as a parameter?

Example: Automatic relevance determination for VAE

$$p(\mathbf{x}|\mathbf{w}) = \int_{\gamma} \int_{\mathbf{z}} p(\mathbf{x}|\mathbf{z}, \gamma, \mathbf{w}) p(\gamma) p(\mathbf{z}) d\gamma d\mathbf{z},$$

$$p(\mathbf{x}|\mathbf{z}, \gamma, \mathbf{w}) = \text{MLP}(\mathbf{x}, \gamma \cdot \mathbf{w}).$$



Weight agnostic neural network

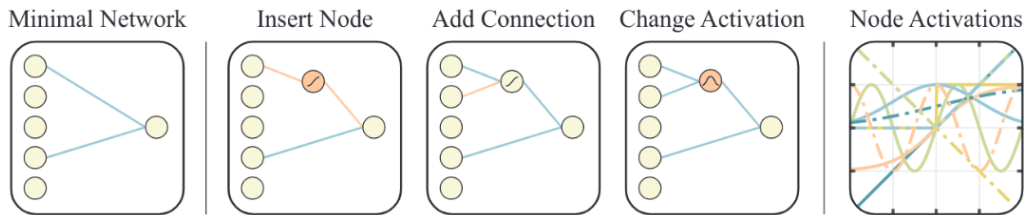


Figure 3: *Operators for Searching the Space of Network Topologies*

Left: A minimal network topology, with input and outputs only partially connected.

Middle: Networks are altered in one of three ways. *Insert Node:* a new node is inserted by splitting an existing connection. *Add Connection:* a new connection is added by connecting two previously unconnected nodes. *Change Activation:* the activation function of a hidden node is reassigned.

Right: Possible activation functions (linear, step, sin, cosine, Gaussian, tanh, sigmoid, absolute value, invert (i.e. negative linear), ReLU) shown over the range $[2, 2]$.

Feature selection

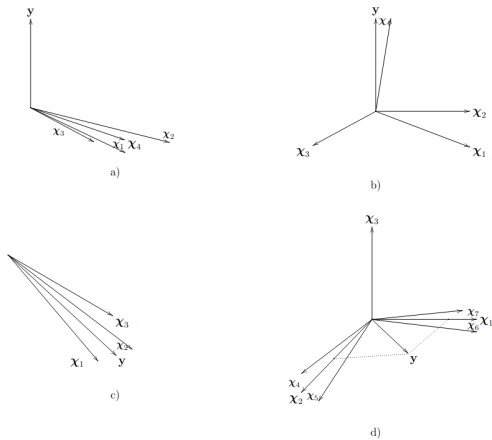


Figure 1: Synthetic test data sets configuration: **a)** inadequate and correlated, **b)** adequate and random, **c)** adequate and redundant, **d)** adequate and correlated.

Note: the problem becomes closer to the structure selection problem with relaxation methods development.

Optimal Brain Damage

The problem of removing irrelevant parameters (pruning) is considered.

Idea: Consider Taylor series for maximum point θ^* :

$$L(\theta^* + \Delta\theta) - L(\theta^*) = -\frac{1}{2}\theta^T H \theta + o(\|\Delta\theta\|^3),$$

where H is Hessian of $-L$.

Diagonalize the Hessian:

$$L(\theta^* + \Delta\theta) \rightarrow \max$$

where

$$\theta_i^* + \Delta\theta_i = 0.$$

Relevance of parameter:

$$\frac{\theta_i^2}{2[H^{-1}]_{i,i}}.$$

Importance estimation for neural network pruning

Basic idea is based on OBD-like Taylor expansion:

$$Q_i = (L(\mathbf{w}) - L(\mathbf{w}|w_i = 0))^2,$$

we can perform first order or second order approximation:

$$Q_i^1 = (\nabla_{w_i} L \cdot w_i)^2, \quad Q_i^2 = (\nabla_{w_i} L \cdot w_i - \frac{1}{2} w_i \mathbf{H}_{i,\cdot} \mathbf{w})^2.$$

Importance estimation for neural network pruning

Group contribution:

$$Q^1 = \left(\sum_i \nabla_{w_i} L \cdot w_i \right)^2.$$

Alternatively we can assign importance of a set of parameters via gating:

$$\mathbf{w} \cdot \gamma, \gamma \in \{0, 1\}, \quad Q^1 = \left(\frac{\partial L}{\partial \gamma} \right) = \left(\sum_i \nabla_{w_i} L \cdot w_i \right)^2.$$

Note on the relation to information theory: Let $h = \frac{\partial L}{\partial \gamma}$. If the model and structure is converged ($E(h)^2 = 0$), then

$$\text{Var}(h) = E(h^2) - (Eh)^2 = Q^1.$$

If L is a log-likelihood, then taking expectation (in variance) across population we get $\text{Var}(h)$ is a Fisher information.

Model generation: example

Adams et al., 2010:

- The problem is to generate Deep belief networks
- The structure Γ is a sequence of adjacent matrices for each layer
- Generate structure using Monte-Carlo with Indian Buffet prior with parameters α, β
- Hyperparameter interpretation: width and sparsity of each layer



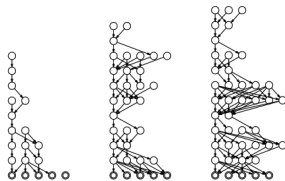
(a) $\alpha = 1, \beta = 1$



(b) $\alpha = \frac{1}{2}, \beta = 1$



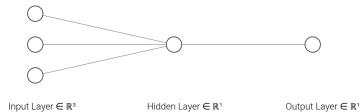
(c) $\alpha = 1, \beta = 2$



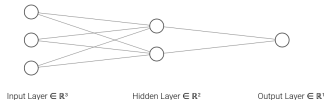
(d) $\alpha = \frac{3}{2}, \beta = 1$

Structure selection example

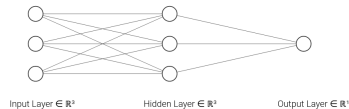
hidden layer dim = 1



hidden layer dim = 2



hidden layer dim = 3



All these models can be represented as $f(x, w) = \sigma \left((w^2)^T \sigma \left((w^1)^T x \right) \right)$
with similar shape of w^1 : $\dim(w^1) = 3 \times 3$.

Structure selection: one-layer network

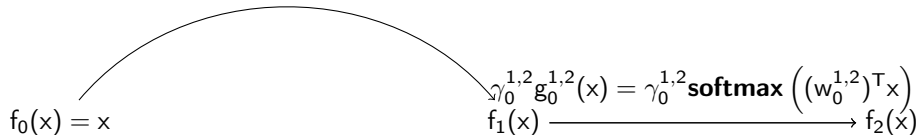
The model f is defined by the **structure** $\Gamma = [\gamma^{0,1}, \gamma^{1,2}]$.

$$\text{Model: } f(x) = \mathbf{softmax} \left((w_0^{1,2})^T f_1(x) \right), \quad f(x) : \mathbb{R}^n \rightarrow [0, 1]^{|\mathbb{Y}|}, \quad x \in \mathbb{R}^n.$$

$$f_1(x) = \gamma_0^{0,1} g_0^{0,1}(x) + \gamma_1^{0,1} g_1^{0,1}(x),$$

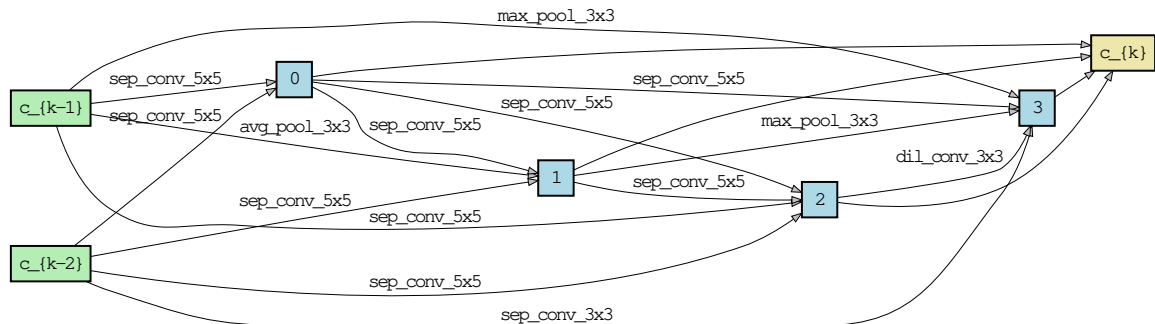
where $w = [w_0^{0,1}, w_1^{0,1}, w_0^{1,2}]^T$ — parameter matrices, $\{g_{0,1}^0, g_{0,1}^1, g_{1,2}^0\}$ — generalized-linear functions, alternatives of layers of the network.

$$\gamma_0^{0,1} g_0^{0,1}(x) = \gamma_0^{0,1} \sigma \left((w_0^{0,1})^T x \right)$$



$$\gamma_1^{0,1} g_1^{0,1}(x) = \gamma_1^{0,1} \sigma \left((w_1^{0,1})^T x \right)$$

Neural architecture search example



Structure selection: neural architecture search space

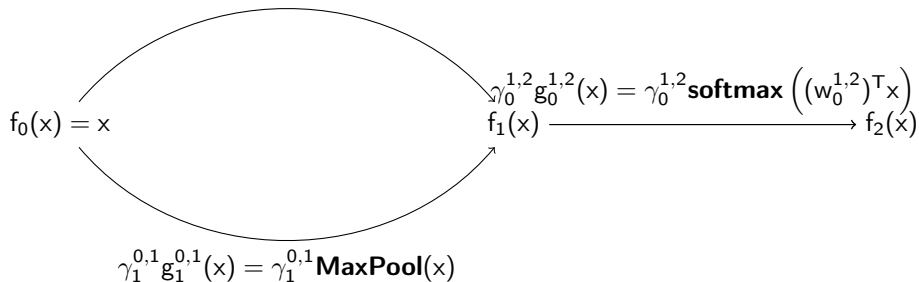
The model f is defined by the **structure** $\Gamma = [\gamma^{0,1}, \gamma^{1,2}]$.

$$\text{Model: } f(x) = \mathbf{softmax} \left((w_0^{1,2})^T f_1(x) \right), \quad f(x) : \mathbb{R}^n \rightarrow [0, 1]^{|Y|}, \quad x \in \mathbb{R}^n.$$

$$f_1(x) = \gamma_0^{0,1} g_0^{0,1}(x) + \gamma_1^{0,1} g_1^{0,1}(x),$$

where $w = [w_0^{0,1}, w_0^{1,2}]^T$ — parameter matrices, $g_{0,1}^0$ is a convolution, $g_{0,1}^1$ is a pooling operation, $g_{1,2}^0$ is a generalized-linear function.

$$\gamma_0^{0,1} g_0^{0,1}(x) = \gamma_0^{0,1} \mathbf{Conv}(x, w_0^{0,1})$$



Deep learning model structure as a graph

Define:

- ① acyclic graph (V, E) ;
- ② for each edge $(j, k) \in E$: a vector primitive differentiable functions $g^{j,k} = [g_0^{j,k}, \dots, g_{K^{j,k}}^{j,k}]$ with length of $K^{j,k}$;
- ③ for each vertex $v \in V$: a differentiable aggregation function $\mathbf{agg}_v$.
- ④ a function $f = f_{|V|-1}$:

$$f_v(w, x) = \mathbf{agg}_v \left(\{ \langle \gamma^{j,k}, g^{j,k} \rangle \circ f_j(x) \mid j \in \text{Adj}(v_k) \} \right), v \in \{1, \dots, |V| - 1\}, \quad f_0(x) = x \quad (1)$$

that is a function from $\mathbb{X}$ into a set of labels $\mathbb{Y}$ for any value of $\gamma^{j,k} \in [0, 1]^{K^{j,k}}$.

Definition

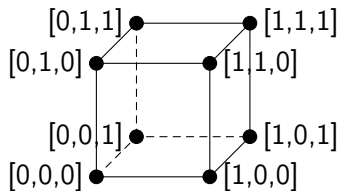
A *parametric set of models* $\mathfrak{F}$ is a graph (V, E) with a set of primitive functions $\{g^{j,k}, (j, k) \in E\}$ and aggregation functions $\{\mathbf{agg}_v, v \in V\}$.

Statement

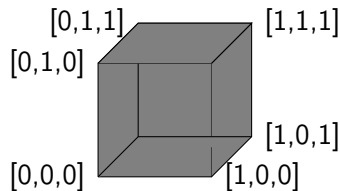
A function $f \in \mathfrak{F}$ is a model for each $\gamma^{j,k} \in [0, 1]^{K^{j,k}}$.

Structure restrictions

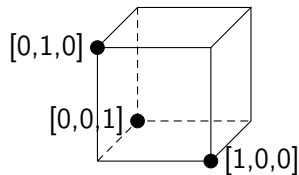
An example of restrictions for structure parameter γ , $|\gamma| = 3$.



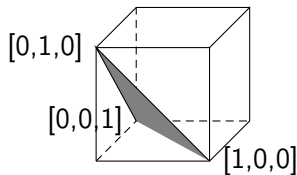
Cube vertices



Cube interior



Simplex vertices

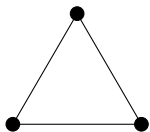


Simplex interior

Prior distribution for the model structure

Every point in a simplex defines a model.

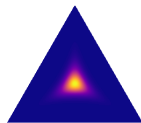
Gumbel-Softmax distribution: $\boldsymbol{\Gamma} \sim \text{GS}(\mathbf{s}, \lambda_{\text{temp}})$



$$\lambda_{\text{temp}} \rightarrow 0$$

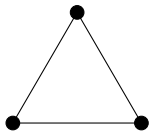


$$\lambda_{\text{temp}} = 0.995$$



$$\lambda_{\text{temp}} = 5.0$$

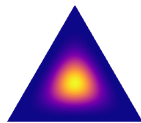
Dirichlet distribution: $\boldsymbol{\Gamma} \sim \text{Dir}(\mathbf{s}, \lambda_{\text{temp}})$



$$\lambda_{\text{temp}} \rightarrow 0$$



$$\lambda_{\text{temp}} = 0.995$$



$$\lambda_{\text{temp}} = 5.0$$

Neural Architecture Search: problem statement

$\mathbf{w}$ are model parameters.

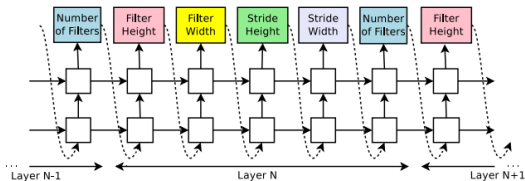
Γ is a structure.

$$\Gamma^* = \arg \max Q(\mathbf{w}^*, \Gamma),$$

$$\mathbf{w}^* = \arg \max L(\mathbf{w}, \Gamma).$$

Neural Architecture Search with Reinforcement Learning

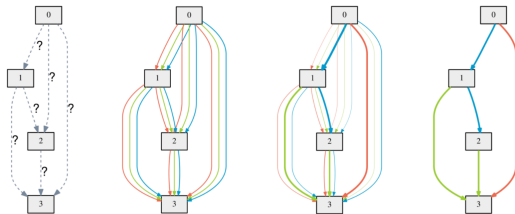
The structure is selected using controller.
The optimization of model parameters is conducted in a loop of structure selection.



DARTS

The model is a multigraph, where edges $[g^e]$ correspond to submodels, vertices $\mathbf{f}_v(\mathbf{x})$ are the results of submodels:

$$\mathbf{f}_v = \langle \gamma, \text{softmax}([g^e(\mathbf{x})]) \rangle.$$



DARTS

Optimization:

$$\Gamma^* = \arg \max Q(\mathbf{w}^*, \Gamma),$$

$$\mathbf{w}^* = \arg \max L(\mathbf{w}, \Gamma).$$

The optimization is done using greedy gradient-like optimization:

$$\nabla_{\Gamma} Q(\mathbf{w}', \Gamma) = \lambda_L \nabla_{\Gamma, \mathbf{w}} L(\mathbf{w}, \Gamma) \nabla_{\mathbf{w}} Q(\Gamma, \mathbf{w}').$$

Performance criteria for structure selection

- Number of parameters
- Number of vertices
- Number of edges
- Complexity of subfunctions

FBNet

$$\min_{\Gamma} \min_w L \cdot \lambda_1 \log \text{LAT}(\Gamma),$$

where LAT is a function of hardware latency of the operations for **target hardware**.

FBNet

Model	#Parameters	#FLOPs	Latency on iPhone X	Latency on Samsung S8	Top-1 acc (%)
FBNet-iPhoneX	4.47M	322M	19.84 ms (target)	23.33 ms	73.20
FBNet-S8	4.43M	293M	27.53 ms	22.12 ms (target)	73.27

Table 5. FBNets searched for different devices.

NAS with complexity control

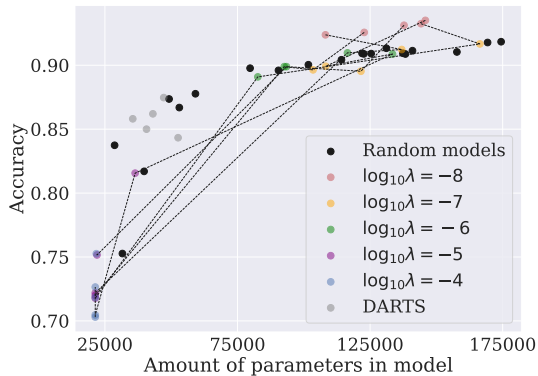
Our proposal

To use a mapping $\gamma(\lambda)$ instead of constant structural parameters $\gamma(\lambda)$, where λ is a regularization term for the loss function:

$$E_{\lambda} \left(\log p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \Gamma(\lambda)) + \lambda \sum_{(i,j)} \langle \text{softmax} \left(\gamma(\lambda)^{(i,j)} \right), \mathbf{n}(\mathbf{g}^{(i,j)}) \rangle \right),$$

where $\mathbf{n}(\mathbf{g}^{(i,j)})$ is a vector of amount of parameters for all the basic functions $\mathbf{g}$.

Example: CIFAR-10



$$E_{\lambda} \left(\log p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \Gamma(\lambda)) + \lambda \sum_{(i,j)} \langle \text{softmax} \left(\gamma(\lambda)^{(i,j)} \right), \mathbf{n}(\mathbf{g}^{(i,j)}) \rangle \right).$$

Why symbolic models for AutoML?¹

- Classical approaches for AutoML usually work in a very limited search space.
- **What if we don't have any knowledge about the model to use?**

¹See Kseniia Petrushina's talk, 2023

Proposed approach

Idea

Evolutionary symbolic search over basic mathematical operations

Process formulation

- ① Given a set of ML tasks $\mathcal{T}$, find optimal algorithm $\alpha^* \in \mathcal{A}$
- ② Quality of α is measured on $\mathcal{T}_{search} \subset \mathcal{T}$. Each search experiment produces candidate algorithm
- ③ The best candidate is chosen on $\mathcal{T}_{select} \subset \mathcal{T}$

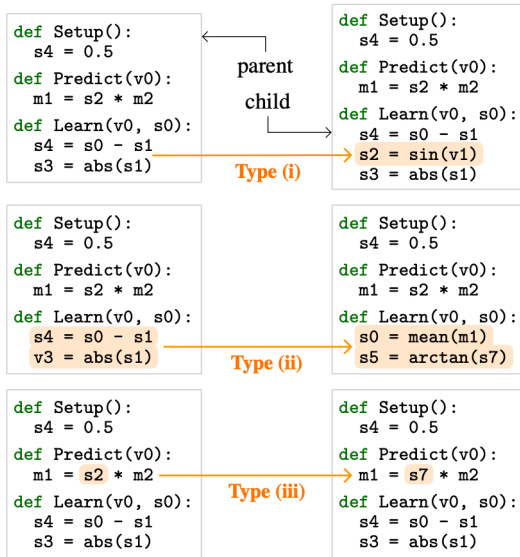
Search space

- The model is decomposed into 3 functions:
 - ▶ Setup: parameters initialization
 - ▶ Predict: make a prediction
 - ▶ Learn: make adjustment in prediction
- Each program has special memory registers for vectors and scalars
- Each program line is constructed by basic arithmetic operation and one or multiple registers
- The objects, labels and predicted labels are stored in the pre-defined registers (v0, s0, s1)

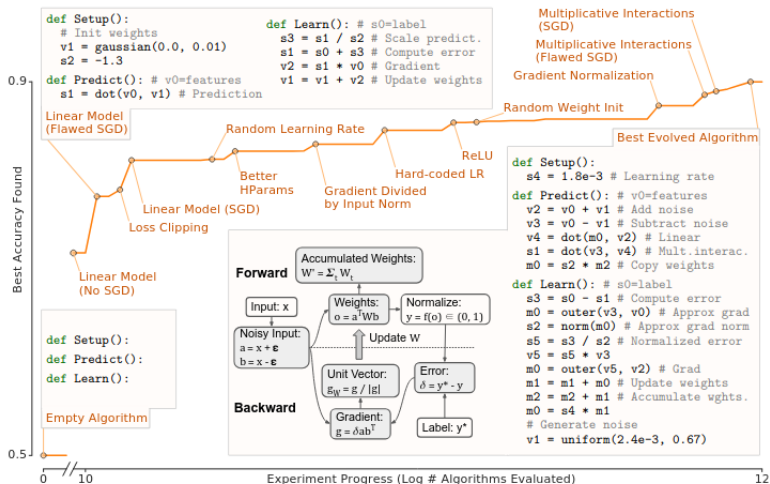
```
# (Setup, Predict, Learn) = input ML algorithm.
# Dtrain / Dvalid = training / validation set.
# sX/vX/mX: scalar/vector/matrix var at address X.
def Evaluate(Setup, Predict, Learn, Dtrain,
Dvalid):
    # Zero-initialize all the variables (sX/vX/mX).
    initialize_memory()
    Setup() # Execute setup instructions.
    for (x, y) in Dtrain:
        v0 = x # x will now be accessible to Predict.
        Predict() # Execute prediction instructions.
        # s1 will now be used as the prediction.
        s1 = Normalize(s1) # Normalize the prediction.
        s0 = y # y will now be accessible to Learn.
        Learn() # Execute learning instructions.

    sum_loss = 0.0
    for (x, y) in Dvalid:
        v0 = x
        Predict() # Only Predict(), not Learn().
        s1 = Normalize(s1)
        sum_loss += Loss(y, s1)
    mean_loss = sum_loss / len(Dvalid)
    # Use validation loss to evaluate the algorithm.
    return mean_loss
```

Evolutionary algorithm: mutation examples



Results



Relative success rate of evolution and random search (RS).

Emerging techniques

Emerging techniques

- Regularization

$$a = x + u, \quad b = x - u, \quad u = U(\alpha, \beta)$$

- Multiplicative interactions

$$o = a^T W b$$

- Gradient normalization

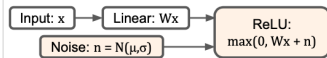
$$g_w = \frac{g}{|g|}; \quad g = \delta a b^T; \quad \delta = y^* - y$$

- Weight averaging

$$W' = \sum_t W_t$$

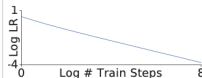
Adaptation to different tasks

```
def Predict():  
    ... # Omitted/irrelevant code  
    # v0=features; m1=weight matrix  
    v6 = dot(m1, v0) # Apply weights  
    # Random vector,  $\mu=-0.5$  and  $\sigma=0.41$   
    v8 = gaussian(-0.5, 0.41)  
    v6 = v6 + v8 # Add it to activations  
    v7 = maximum(v9, v6) # ReLU, v9 $\approx$ 0  
    ...
```



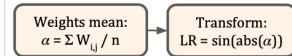
(a) Adaptation to few examples.

```
def Setup():  
    # LR = learning rate  
    s2 = 0.37 # Init. LR  
    ...  
def Learn():  
    # Decay LR  
    s2 = arctan(s2)  
    ...
```



(b) Adaptation to fast training.

```
def Learn():  
    s3 = mean(m1) # m1 is the weights.  
    s3 = abs(s3)  
    s3 = sin(s3)  
    # From here down, s3 is used as  
    # the learning rate.  
    ...
```



(c) Adaptation to multiple classes.

Adaptations to different task types.

Reference

- Bishop C. M., Nasrabadi N. M. Pattern recognition and machine learning. – New York : springer, 2006. – Т. 4. – №. 4. – С. 738.
- Mansinghka V. et al. Structured priors for structure learning //arXiv preprint arXiv:1206.6852. – 2012.
- Варфоломеева А. А. Методы структурного обучения в задаче обобщения структур прогностических моделей, магистерская диссертация.
- LeCun Y., Denker J., Solla S. Optimal brain damage //Advances in neural information processing systems. – 1989. – Т. 2.
- Han S. et al. Learning both weights and connections for efficient neural network //Advances in neural information processing systems. – 2015. – Т. 28
- Karaletsos, Theofanis, and Gunnar Rätsch. "Automatic relevance determination for deep generative models." arXiv preprint arXiv:1505.07765 (2015).
- Gaier, Adam, and David Ha. "Weight agnostic neural networks." Advances in neural information processing systems 32 (2019).
- Katrutsa, Alexandr, and Vadim Strijov. "Comprehensive study of feature selection methods to solve multicollinearity problem according to evaluation criteria." Expert Systems with Applications 76 (2017): 1-11.
- Molchanov, Pavlo, et al. "Importance estimation for neural network pruning." Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2019.
- Adams R. P., Wallach H., Ghahramani Z. Learning the structure of deep sparse graphical models //Proceedings of the thirteenth international conference on artificial intelligence and statistics. – JMLR Workshop and Conference Proceedings, 2010. – С. 1-8.
- Jang E., Gu S., Poole B. Categorical reparameterization with gumbel-softmax //arXiv preprint arXiv:1611.01144. – 2016.
- Zoph B., Le Q. V. Neural architecture search with reinforcement learning //arXiv preprint arXiv:1611.01578. – 2016.
- Бахтеев О. Ю. 2020. Байесовский выбор субоптимальной структуры модели глубокого обучения. Диссертация.
- Liu H., Simonyan K., Yang Y. Darts: Differentiable architecture search //arXiv preprint arXiv:1806.09055. – 2018.
- Yakovlev K. D. et al. Neural Architecture Search with Structure Complexity Control //International Conference on Analysis of Images, Social Networks and Texts. – Springer, Cham, 2022. – С. 207-219.
- Wu B. et al. Fbnet: Hardware-aware efficient convnet design via differentiable neural architecture search //Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. – 2019. – С. 10734-10742.
- Real, Esteban, et al. "Automl-zero: Evolving machine learning algorithms from scratch." International conference on machine learning. PMLR, 2020.

Organizational issues

- Technical meeting: next week, regular time
- Format: each team shows the basic code. You should have an idea how this will be transformed into demo.
- The draft version of blog-post and documentation will be checked in offline-mode.
- All the presented materials must be stored at the github
 - ▶ for blog-post, you can put read-only link for the overleaf if you write the post here.
 - ▶ Time-limit: 10 minutes.